

CBBA 訊息介面說明 (swarm_interfaces v1.0)

uav_cbba · 2026-10-05

1. CBBA 需要交換什麼

CBBA 中每台載具 i 會維護三份資訊，訊息就是用來把它們傳給其他載具：

符號	意義	對應的訊息欄位
y_{ij}	載具 i 目前知道的「任務 j 最高出價」	<code>Bid.score</code>
z_{ij}	載具 i 目前知道的「任務 j 得標者」	<code>Bid.winner_id</code>
s_{ik}	載具 i 最後一次取得載具 k 資訊的時間	<code>AgentStamp.stamp</code>

- y 和 z 讓大家知道誰出了多少、誰得標。
- s 用來判斷誰的資訊比較新。兩台對同一個任務說法不同時，衝突消解規則（Choi 2009 的 17 條）要靠時間戳決定採信誰。

除此之外，大家還需要一份共用的「任務清單」，否則出價對不上任務。

2. 訊息架構

`/swarm/tasks`
`/swarm/cbba`

`— Task`
`— CBBAMessage`
 `└ Bid[]`
 `└ AgentStamp[]`

`—— 任務清單（所有載具都要先知道有哪些任務）`
`—— 某一台載具的出價表`
`—— 每個 OPEN 任務一筆： $y, z$`
`—— 每台已知的載具一筆： $s$`

- 任務和出價分成兩個話題：**任務很少變動，只在建立和結束時發送；出價則一直在變。兩者的傳送需求不同，所以 QoS 分開設定（見第 4 節）。
- Bid 和 AgentStamp 是子訊息：**它們不會單獨發送，一定包在 CBBAMessage 裡。一則 CBBAMessage 就是某台載具的完整出價狀態。

3. 各訊息欄位

3-1 Task：任務定義（ /swarm/tasks ）

欄位	型別	說明
task_id	uint32	建立者 agent_id × 65536 + 流水號。各台自己編號，不需要協調也不會撞號
type	uint8	空中偵察 1、地面處置 2、巡邏 3。搭配相容表決定誰能出價
position	geometry_msgs/Point	map 座標系（ENU），單位公尺
created	builtin_interfaces/Time	任務發現或建立的時間
deadline_sec	float32	期限，從 created 起算的秒數（軟限制，見第 6 節）
value	float32	任務價值，也是出價分數的上限
duration_sec	float32	預估的現場執行時間，用來計算懸停耗電
status	uint8	OPEN 0、DONE 1、CANCELLED 2
status_stamp	builtin_interfaces/Time	狀態最後一次改變的時間

- 任務完成或取消時，會用同一個 task_id 再發一次，只是 status 不同。各台收到後就停止對它出價。

相容表（不相容的組合，出價固定為 0）：

	空中偵察	地面處置	巡邏
無人機	✓	0	0
V60	0	✓	✓

3-2 CBBAMessage：某台載具的出價表（ /swarm/cbba ）

欄位	型別	說明
header.stamp	Time	送出時間
sender_id	uint8	傳送者的 agent_id
sender_type	uint8	UAV 1、UGV 2（見第 6 節）
seq	uint32	傳送者自己的流水號，用來丟掉舊訊息（見第 6 節）
bids[]	Bid[]	每個 OPEN 任務一筆
stamps[]	AgentStamp[]	傳送者以外、每台已知的載具一筆

- 每次都送完整的出價表，而不是只送有變動的部分。掉了幾則也沒關係，下一則就能補齊全部狀態。
- stamps 不含傳送者自己：接收者收到這則訊息時，就等於取得了傳送者的最新資訊，時間就是這則訊息的送出時間，不需要另外附上。

3-3 Bid：單一任務的出價

欄位	型別	說明
task_id	uint32	任務 ID
score	float32	傳送者目前知道的最高分 y。沒有人得標時為 0
winner_id	uint8	傳送者目前知道的得標者 z。0 表示無人

注意：Bid 傳的是「傳送者**所知道**的最高出價和得標者」，不一定是傳送者自己的出價。例如 uav1 知道任務 5 是 uav2 以 70 分得標，它送出的就是 {task 5, 70, 2}。資訊就是這樣一台傳一台擴散出去的。

3-4 AgentStamp：資訊時間戳

欄位	型別	說明
agent_id	uint8	載具編號
stamp	builtin_interfaces/Time	傳送者最後一次取得該載具資訊的時間

用途舉例：uav1 和 uav3 對任務 5 的得標者說法不同，一個說是 uav2，一個說是無人。這時就比較兩者對 uav2 的時間戳，較新的那一方說了算。

3-5 編號規則

載具	agent_id	話題命名空間
無人機 N (N = 1~9)	N	/uavN/ ...，PX4 話題為 /uavN/fmu/ ...
Ghost V60 機器狗	50 (之後多隻依序為 51、52...)	/v60/ ...
地面站	不參與出價	—

- agent_id = 0 保留給「無人得標」。
- 群體共用的話題放在 /swarm/ 下，不加载具命名空間。

4. QoS 設計

	/swarm/tasks	/swarm/cbba
可靠性	reliable：遺失會重傳	best effort：遺失不重傳
持久性	transient_local：保留最近 100 則，新加入的節點也收得到	volatile：只給當下在線的節點
發送時機	建立時一次，狀態改變時一次	出價表改變時立即送，另外每 200 ms 重送

理由：

- 任務不能遺失：核心收到「沒見過的任務」的出價會直接略過，所以任務一定要先送到。用 reliable 加 transient_local，晚啟動或斷線重連的節點也能拿到完整的任務清單。

- **出價不需要可靠傳輸**：出價隨時在變，重傳舊的出價沒有意義。每次送的都是完整狀態，再加上 200 ms 週期重送，掉的訊息自然會被下一則補上。如果改用 reliable，弱網時重傳反而會塞住新的資訊。

5. 範例：一輪資訊交換

場景：uav1 和 uav2 都看得到任務 65537（建立者 1、流水號 1）。

① 地面站在 /swarm/tasks 發布

```
Task{task_id=65537, type=AIR_RECON, position=(10,5,5), deadline=60s, value=80, status=OPEN}
```

② uav1 算出分數 62，在 /swarm/cbba 送出

```
CBBAMessage{sender=1, seq=12,
             bids=[{65537, 62.0, winner=1}],
             stamps=[{agent=2, stamp=t0}]}
```

③ uav2 算出分數 70，比 62 高，於是改成自己得標並送出

```
CBBAMessage{sender=2, seq=8,
             bids=[{65537, 70.0, winner=2}],
             stamps=[{agent=1, stamp=t1}]}
```

④ uav1 收到後，依消解規則接受 uav2 得標，退出這個任務。

之後雙方送的 Bid 都是 {65537, 70.0, winner=2} → 達成共識

6. 重點欄位的來源與用意

6-1 deadline_sec (Task, float32)

意義：任務的期限，從 created（任務建立時間）起算的秒數。

來源：

- 「任務有時間限制」的概念來自 **Velhal 2022** 的時空任務：入侵者會在某個時間抵達邊界，防守者必須在那之前趕到。
- 在 Velhal 的論文裡這是**硬限制**：遲到就等於防守失敗，路徑直接不可行。本研究把它改成**軟限制**。

怎麼用在公式裡：時間成本 $L_t = \text{到達時間} \div \text{剩餘期限}$ 。

情況	L_t	效果
來得及	< 1	成本正常
會遲到	> 1	成本變高，分數降低
已經過期	剩餘期限被夾在 1 秒	成本很大，但仍有限值

分數公式是 $\text{value} / (1 + \Delta C / c_{\text{ref}})$ ，不論成本多大，分數都是正的，所以**逾期的任務還是可以被接**。

為什麼用軟限制：

- 火場的任務遲到了還是要處理。如果用硬限制，沒有人能準時趕到的火點就會**沒人接**，這比晚一點到更糟。
- 用軟限制時，遲到的機子分數會被壓低：有能準時到的機子就交給它，沒有的話，至少還有人去。

為什麼用「建立時間 + 秒數」，而不是直接給截止時刻：

- 寫任務時直接填「60 秒內」比較直覺，例如火點 60 s、巡檢點 300 s、地面處置 120 s。
- 各機用同一個 `created` 當基準，就能算出剩餘時間。前提是各機時鐘已同步（模擬時在同一台主機，實機用 `chrony`）。

6-2 `sender_type` (`CBBA`Message, `uint8`)

意義：傳送者是哪一類載具，UAV = 1、UGV = 2。

來源：本專案是異質機隊（無人機加 V60 機器狗），規格中為了區分機種而加入，不是來自論文。

用意：

- 收到出價時，可以知道對方是無人機還是機器狗，方便監控、記錄和除錯。
- 保留給之後的異質邏輯使用，例如依機種統計，或在地面站畫面上分開顯示。

目前狀態：`CBBA` 核心（`cbba_agent.cpp`）中，`sender_type` 只在送出訊息時填入，收到時沒有被使用。

- 誰能接哪種任務，是各機用**自己的**類型對照相容表來決定的，例如無人機對地面任務直接出價 0。
- 所以不需要看對方的類型，也不影響分配結果。

6-3 `seq` (`CBBA`Message, `uint32`)

意義：傳送者自己的訊息流水號，每送一則就加 1。

來源：網路通訊的通用做法（sequence number），不是來自 `CBBA` 論文。

程式中的做法（`cbba_agent.cpp` 的 `onMessage`）：

```
if (msg.seq ≤ 上次收到這台的 seq) {  
    return false;    // 舊的或重複的訊息，直接丟掉  
}
```

為什麼需要：

- `/swarm/cbba` 用的是 best effort 傳輸，經過 mesh 網路時，訊息可能**亂序**（後送的先到）或**重複**。
- 如果舊訊息比新訊息晚到，又被拿去處理，就會用過時的出價蓋掉新的出價，讓共識來回震盪、收斂變慢。
- 有了 `seq`，每台只處理比上次更新的訊息，就能保證看到的是對方的最新狀態。

要注意的情況：

- 如果某台的 `CBBA` 節點**重新啟動**，它的 `seq` 會從 1 重新開始。
- 其他機子還記得它之前的 `seq`（例如 500），會把它重啟後送出的訊息全部當成舊訊息丟掉，直到 `seq` 再超過 500 為止。以每 200 ms 重送一次計算，大約要等 100 秒。
- 目前程式還沒處理這個情況。之後寫 ROS 節點時，可以在 `seq` 突然變小很多、或很久沒收到這台的訊息時，重新接受它的 `seq`。

uint32 上限約 43 億，以每 200 ms 一則計算，要 27 年才會溢位，不需要擔心。

6-4 總結

欄位	來源	用意	目前狀態
deadline_sec	Velhal 2022 的時空任務，改為軟限制	讓急的任務優先；遲到的任務仍有人接	已用於時間成本 Lt
sender_type	本專案（異質機隊）	識別載具類型，供監控與擴充	只填入，核心未使用
seq	網路通訊的通用做法	丟掉亂序或重複的舊訊息	已使用；節點重啟的情況尚未處理

參考文獻

- H.-L. Choi, L. Brunet, J. P. How, "Consensus-Based Decentralized Auctions for Robust Task Allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, 2009.
- S. Velhal, S. Sundaram, N. Sundararajan, "A Decentralized Multirobot Spatiotemporal Multitask Assignment Approach for Perimeter Defense," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3085–3096, 2022.